Event ends in 2 days 2 hours 54 minutes.

This implementation section brings together all Frontend phase specifications into a complete, production-ready React application using AWS Cloudscape Design System. You'll implement the design system, component architecture, performance optimizations, and accessibility features with react-design-systems-mcp server guidance.

Implementation Overview

Based on your Frontend specifications, you'll implement:

- **AWS Cloudscape Design System** with comprehensive UI components and design tokens
- **React component architecture** with TypeScript and modern patterns
- **Performance optimization** with PWA features and Core Web Vitals compliance
- **Accessibility and testing** with WCAG 2.1 AA compliance and comprehensive test coverage

Step 1: AWS Cloudscape Setup and MCP Integration

Implement AWS Cloudscape with react-design-systems-mcp server guidance:

~~~
Set up AWS Cloudscape and configure the react-design-systems-mcp server:

**Step 1: Configure React Design Systems MCP Server**
Add to your `.kiro/settings/mcp.json`:

```json
{
  "mcpServers": {
    "react-design-systems": {
      "command": "uvx",
      "args": ["react-design-systems-mcp@latest"],
      "disabled": false,
      "autoApprove": [
        "get-component-info",
        "get-design-tokens",
        "get-best-practices",
        "validate-accessibility"
      ],
      "disabledTools": []
    }
  }
}
```

**Step 2: Install AWS Cloudscape Dependencies**
```bash
# Install AWS Cloudscape Design System
npm install @cloudscape-design/components @cloudscape-design/global-styles
npm install @cloudscape-design/design-tokens @cloudscape-design/collection-hooks

# Install additional dependencies for enhanced functionality
npm install @cloudscape-design/code-view @cloudscape-design/chat-components
npm install @cloudscape-design/board-components

# Install development dependencies
npm install --save-dev @cloudscape-design/jest-preset
npm install --save-dev @cloudscape-design/test-utils
```

**Step 3: Configure Cloudscape Theme and Global Styles**
Create `src/styles/cloudscape-theme.ts`:

```typescript
import { applyMode, Mode } from '@cloudscape-design/global-styles';

export interface CloudscapeTheme {
  mode: Mode;
  density: 'comfortable' | 'compact';
  motionDisabled: boolean;
}

export const defaultTheme: CloudscapeTheme = {
  mode: Mode.Light,
  density: 'comfortable',
  motionDisabled: false
};

export const applyCloudscapeTheme = (theme: CloudscapeTheme) => {
  // Apply Cloudscape mode (light/dark)
  applyMode(theme.mode);

  // Apply density mode
  document.documentElement.setAttribute('data-awsui-density-mode', theme.density);

  // Apply motion preferences
  if (theme.motionDisabled) {
    document.documentElement.setAttribute('data-awsui-motion', 'disabled');
  } else {
    document.documentElement.removeAttribute('data-awsui-motion');
  }
};

// Design tokens for custom components
export const designTokens = {
  colors: {
    primary: {
      50: '#f0f9ff',
      100: '#e0f2fe',
      500: '#0ea5e9',
      600: '#0284c7',
      700: '#0369a1'
    },
    semantic: {
      success: '#16a34a',
~~~

~~~
      warning: '#d97706',
      error: '#dc2626',
      info: '#2563eb'
    }
  },
  spacing: {
    xs: '4px',
    s: '8px',
    m: '16px',
    l: '24px',
    xl: '32px',
    xxl: '40px'
  },
  typography: {
    fontFamily: 'Amazon Ember, Helvetica, Arial, sans-serif',
    fontSize: {
      xs: '12px',
      s: '14px',
      m: '16px',
      l: '18px',
      xl: '20px',
      xxl: '24px'
    },
    fontWeight: {
      normal: 400,
      medium: 500,
      bold: 700
    }
  },
  borderRadius: {
    s: '4px',
    m: '8px',
    l: '12px'
  }
};
```

**Step 4: Create Cloudscape App Layout**
Create `src/components/layout/AppLayout.tsx`:

```typescript
import React, { useState } from 'react';
import {
  AppLayout as CloudscapeAppLayout,
  SideNavigation,
  TopNavigation,
  BreadcrumbGroup,
  HelpPanel,
  SplitPanel
} from '@cloudscape-design/components';
import { useAuthStore } from '@/stores/authStore';
import { useThemeStore } from '@/stores/themeStore';

interface AppLayoutProps {
  children: React.ReactNode;
  navigationOpen?: boolean;
  toolsOpen?: boolean;
  splitPanelOpen?: boolean;
}

export const AppLayout: React.FC<AppLayoutProps> = ({
  children,
  navigationOpen = true,
  toolsOpen = false,
  splitPanelOpen = false
}) => {
  const { user, logout } = useAuthStore();
  const { theme, toggleTheme } = useThemeStore();

  const [navigationState, setNavigationState] = useState({
    open: navigationOpen,
    activeHref: window.location.pathname
  });

  const navigationItems = [
    {
      type: 'section',
      text: 'Chat',
      items: [
        { type: 'link', text: 'Conversations', href: '/conversations' },
        { type: 'link', text: 'New Chat', href: '/chat/new' }
      ]
    },
    {
      type: 'section',
      text: 'Management',
      items: [
        { type: 'link', text: 'Profile', href: '/profile' },
        { type: 'link', text: 'Settings', href: '/settings' }
      ]
    }
  ];

  const topNavigationUtilities = [
    {
      type: 'button',
      text: theme.mode === 'light' ? 'Dark mode' : 'Light mode',
      onClick: toggleTheme,
      iconName: theme.mode === 'light' ? 'moon' : 'sun'
    },
    {
      type: 'menu-dropdown',
      text: user?.name || 'User',
      description: user?.email,
      iconName: 'user-profile',
      items: [
        { id: 'profile', text: 'Profile', href: '/profile' },
        { id: 'settings', text: 'Settings', href: '/settings' },
        { id: 'logout', text: 'Sign out', onClick: logout }
      ]
~~~

~~~~
    }
  ];

  return (
    <CloudscapeAppLayout
      headerSelector="#top-navigation"
      navigation={
        <SideNavigation
          header={{ text: 'MyCoolChatbot-KiroSpec', href: '/' }}
          items={navigationItems}
          activeHref={navigationState.activeHref}
          onFollow={(event) => {
            if (!event.detail.external) {
              event.preventDefault();
              setNavigationState({
                ...navigationState,
                activeHref: event.detail.href
              });
              // Handle navigation with your router
            }
          }}
        />
      }
      navigationOpen={navigationState.open}
      onNavigationChange={({ detail }) =>
        setNavigationState({ ...navigationState, open: detail.open })
      }
      tools={
        <HelpPanel
          header={<h2>Help</h2>}
          footer={
            <div>
              <h3>Learn more</h3>
              <ul>
                <li><a href="/docs" target="_blank">User Guide</a></li>
                <li><a href="/api-docs" target="_blank">API Documentation</a></li>
                <li><a href="/support" target="_blank">Support</a></li>
              </ul>
            </div>
          }
        >
          <p>
            Get help and information about using the ChatBot application.
            Use the navigation menu to access different features.
          </p>
        </HelpPanel>
      }
      toolsOpen={toolsOpen}
      splitPanel={
        <SplitPanel header="Debug Information">
          <div>
            <h3>Application State</h3>
            <pre>{JSON.stringify({ user: user?.id, theme }, null, 2)}</pre>
          </div>
        </SplitPanel>
      }
      splitPanelOpen={splitPanelOpen}
      content={children}
    />
  );
};

// Top Navigation Component
export const TopNavigation: React.FC = () => {
  const { user } = useAuthStore();
  const { theme, toggleTheme } = useThemeStore();

  return (
    <div id="top-navigation">
      <TopNavigation
        identity={{
          href: '/',
          title: 'MyCoolChatbot-KiroSpec',
          logo: { src: '/logo.svg', alt: 'MyCoolChatbot-KiroSpec' }
        }}
        utilities={[
          {
            type: 'button',
            text: theme.mode === 'light' ? 'Dark mode' : 'Light mode',
            onClick: toggleTheme,
            iconName: theme.mode === 'light' ? 'moon' : 'sun'
          },
          {
            type: 'menu-dropdown',
            text: user?.name || 'User',
            description: user?.email,
            iconName: 'user-profile',
            items: [
              { id: 'profile', text: 'Profile' },
              { id: 'settings', text: 'Settings' },
              { id: 'logout', text: 'Sign out' }
            ]
          }
        ]}
      />
    </div>
  );
};
```
~~~~

Step 2: Chat Interface with Cloudscape ComponentsImplement

the chat interface using AWS Cloudscape chat components:

~~~~
Create comprehensive chat interface with Cloudscape components:

**Chat Interface Component (src/components/chat/ChatInterface.tsx)**:
~~~~

```typescript
import React, { useState, useRef, useEffect } from 'react';
import {
  Container,
  Header,
  SpaceBetween,
  Button,
  Input,
  Box,
  Alert,
  Spinner,
  Badge,
  StatusIndicator
} from '@cloudscape-design/components';
import { ChatBubble } from '@cloudscape-design/chat-components';
import { useChatStore } from '@/stores/chatStore';
import { useAIService } from '@/hooks/useAIService';
import { Message, Conversation } from '@/types';

interface ChatInterfaceProps {
  conversationId?: string;
  onNewConversation?: (conversation: Conversation) => void;
}

export const ChatInterface: React.FC<ChatInterfaceProps> = ({
  conversationId,
  onNewConversation
}) => {
  const [message, setMessage] = useState('');
  const [isTyping, setIsTyping] = useState(false);
  const messagesEndRef = useRef<HTMLDivElement>(null);

  const {
    currentConversation,
    messages,
    sendMessage,
    isLoading,
    error
  } = useChatStore();

  const { generateResponse } = useAIService();

  const scrollToBottom = () => {
    messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' });
  };

  useEffect(() => {
    scrollToBottom();
  }, [messages]);

  const handleSendMessage = async () => {
    if (!message.trim() || isLoading) return;

    const userMessage = message;
    setMessage('');
    setIsTyping(true);

    try {
      // Send user message
      await sendMessage(userMessage, 'user');

      // Generate AI response
      const aiResponse = await generateResponse(userMessage, messages);
      await sendMessage(aiResponse, 'assistant');
    } catch (error) {
      console.error('Failed to send message:', error);
    } finally {
      setIsTyping(false);
    }
  };

  const handleKeyPress = (event: React.KeyboardEvent) => {
    if (event.key === 'Enter' && !event.shiftKey) {
      event.preventDefault();
      handleSendMessage();
    }
  };

  return (
    <Container
      header={
        <Header
          variant="h2"
          description={currentConversation?.title || 'New conversation'}
          actions={
            <SpaceBetween direction="horizontal" size="xs">
              <Badge color="blue">
                {messages.length} messages
              </Badge>
              <StatusIndicator type={isLoading ? "loading" : "success"}>
                {isLoading ? 'Processing...' : 'Ready'}
              </StatusIndicator>
            </SpaceBetween>
          }
        >
          Chat Interface
        </Header>
      }
    >
      <SpaceBetween size="m">
        {error && (
          <Alert type="error" dismissible>
            {error}
          </Alert>
        )}

        <Box padding="s">
          <div style={{ height: '500px', overflowY: 'auto', padding: '16px' }}>
            <SpaceBetween size="s">
```

~~~
              {messages.map((msg) => (
                <ChatMessage key={msg.id} message={msg} />
              ))}

              {isTyping && (
                <Box>
                  <ChatBubble
                    variant="incoming"
                    content={
                      <SpaceBetween direction="horizontal" size="xs">
                        <Spinner size="small" />
                        <span>AI is typing...</span>
                      </SpaceBetween>
                    }
                  />
                </Box>
              )}
            </SpaceBetween>
            <div ref={messagesEndRef} />
          </div>
        </Box>

        <SpaceBetween direction="horizontal" size="s">
          <Input
            value={message}
            onChange={({ detail }) => setMessage(detail.value)}
            onKeyDown={handleKeyPress}
            placeholder="Type your message..."
            disabled={isLoading}
            ariaLabel="Chat message input"
          />
          <Button
            variant="primary"
            onClick={handleSendMessage}
            disabled={!message.trim() || isLoading}
            ariaLabel="Send message"
          >
            Send
          </Button>
        </SpaceBetween>
      </SpaceBetween>
    </Container>
  );
};

// Individual Chat Message Component
interface ChatMessageProps {
  message: Message;
}

const ChatMessage: React.FC<ChatMessageProps> = ({ message }) => {
  const isUser = message.type === 'user';

  return (
    <Box>
      <ChatBubble
        variant={isUser ? 'outgoing' : 'incoming'}
        content={
          <SpaceBetween size="xs">
            <div>{message.content}</div>
            {message.metadata && (
              <Box fontSize="body-s" color="text-status-inactive">
                {new Date(message.timestamp).toLocaleTimeString()}
                {message.metadata.tokens && ` • ${message.metadata.tokens} tokens`}
              </Box>
            )}
          </SpaceBetween>
        }
      />
    </Box>
  );
};
```

**Conversation List Component (src/components/chat/ConversationList.tsx)**:
```typescript
import React from 'react';
import {
  Table,
  Header,
  Button,
  SpaceBetween,
  Box,
  Badge,
  Link,
  CollectionPreferences,
  Pagination,
  TextFilter
} from '@cloudscape-design/components';
import { useChatStore } from '@/stores/chatStore';
import { Conversation } from '@/types';

export const ConversationList: React.FC = () => {
  const {
    conversations,
    deleteConversation,
    isLoading
  } = useChatStore();

  const [selectedItems, setSelectedItems] = React.useState<Conversation[]>([]);
  const [preferences, setPreferences] = React.useState({
    pageSize: 10,
    visibleContent: ['title', 'messageCount', 'lastActivity', 'actions']
  });

  const columnDefinitions = [
    {
      id: 'title',
~~~

```
      header: 'Conversation',
      cell: (item: Conversation) => (
        <Link href={`/chat/${item.id}`}>
          {item.title || 'Untitled Conversation'}
        </Link>
      ),
      sortingField: 'title'
    },
    {
      id: 'messageCount',
      header: 'Messages',
      cell: (item: Conversation) => (
        <Badge color="blue">{item.metadata.totalMessages}</Badge>
      ),
      sortingField: 'metadata.totalMessages'
    },
    {
      id: 'lastActivity',
      header: 'Last Activity',
      cell: (item: Conversation) => (
        <Box color="text-status-inactive">
          {new Date(item.metadata.lastActivity).toLocaleDateString()}
        </Box>
      ),
      sortingField: 'metadata.lastActivity'
    },
    {
      id: 'actions',
      header: 'Actions',
      cell: (item: Conversation) => (
        <SpaceBetween direction="horizontal" size="xs">
          <Button
            variant="link"
            onClick={() => window.location.href = `/chat/${item.id}`}
          >
            Open
          </Button>
          <Button
            variant="link"
            onClick={() => deleteConversation(item.id)}
          >
            Delete
          </Button>
        </SpaceBetween>
      )
    }
  ];

  return (
    <Table
      columnDefinitions={columnDefinitions}
      items={conversations}
      loading={isLoading}
      selectedItems={selectedItems}
      onSelectionChange={({ detail }) => setSelectedItems(detail.selectedItems)}
      selectionType="multi"
      ariaLabels={{
        selectionGroupLabel: "Items selection",
        allItemsSelectionLabel: ({ selectedItems }) =>
          `${selectedItems.length} ${
            selectedItems.length === 1 ? "item" : "items"
          } selected`,
        itemSelectionLabel: ({ selectedItems }, item) => {
          const isItemSelected = selectedItems.filter(
            i => i.id === item.id
          ).length;
          return `${item.title} is ${
            isItemSelected ? "" : "not "
          }selected`;
        }
      }}
      header={
        <Header
          counter={`(${conversations.length})`}
          actions={
            <SpaceBetween direction="horizontal" size="xs">
              <Button
                disabled={selectedItems.length === 0}
                onClick={() => {
                  selectedItems.forEach(item => deleteConversation(item.id));
                  setSelectedItems([]);
                }}
              >
                Delete selected
              </Button>
              <Button variant="primary" href="/chat/new">
                New conversation
              </Button>
            </SpaceBetween>
          }
        >
          Conversations
        </Header>
      }
      preferences={
        <CollectionPreferences
          title="Preferences"
          confirmLabel="Confirm"
          cancelLabel="Cancel"
          preferences={preferences}
          onConfirm={({ detail }) => setPreferences(detail)}
          pageSizePreference={{
            title: "Page size",
            options: [
              { value: 10, label: "10 conversations" },
              { value: 20, label: "20 conversations" },
              { value: 50, label: "50 conversations" }
            ]
          }}
```

~~~
          visibleContentPreference={{
            title: "Select visible content",
            options: [
              { id: "title", label: "Conversation title" },
              { id: "messageCount", label: "Message count" },
              { id: "lastActivity", label: "Last activity" },
              { id: "actions", label: "Actions" }
            ]
          }}
        />
      }
      pagination={
        <Pagination
          currentPageIndex={1}
          pagesCount={Math.ceil(conversations.length / preferences.pageSize)}
        />
      }
      filter={
        <TextFilter
          filteringPlaceholder="Find conversations"
          filteringText=""
        />
      }
    />
  );
};
```
~~~

Step 3: State Management with Zustand and Cloudscape IntegrationImpleme

nt comprehensive state management with Zustand:

~~~
Create state management stores with Cloudscape integration:

**Theme Store (src/stores/themeStore.ts)**:
```typescript
import { create } from 'zustand';
import { persist } from 'zustand/middleware';
import { Mode } from '@cloudscape-design/global-styles';
import { CloudscapeTheme, applyCloudscapeTheme, defaultTheme } from '@/styles/cloudscape-theme';

interface ThemeState {
  theme: CloudscapeTheme;
  setTheme: (theme: Partial<CloudscapeTheme>) => void;
  toggleTheme: () => void;
  setDensity: (density: 'comfortable' | 'compact') => void;
  toggleMotion: () => void;
}

export const useThemeStore = create<ThemeState>()(
  persist(
    (set, get) => ({
      theme: defaultTheme,

      setTheme: (newTheme) => {
        const updatedTheme = { ...get().theme, ...newTheme };
        set({ theme: updatedTheme });
        applyCloudscapeTheme(updatedTheme);
      },

      toggleTheme: () => {
        const currentMode = get().theme.mode;
        const newMode = currentMode === Mode.Light ? Mode.Dark : Mode.Light;
        get().setTheme({ mode: newMode });
      },

      setDensity: (density) => {
        get().setTheme({ density });
      },

      toggleMotion: () => {
        const currentMotion = get().theme.motionDisabled;
        get().setTheme({ motionDisabled: !currentMotion });
      }
    }),
    {
      name: 'cloudscape-theme',
      onRehydrateStorage: () => (state) => {
        if (state?.theme) {
          applyCloudscapeTheme(state.theme);
        }
      }
    }
  )
);
```

**Chat Store (src/stores/chatStore.ts)**:
```typescript
import { create } from 'zustand';
import { persist } from 'zustand/middleware';
import { Message, Conversation, User } from '@/types';
import { chatAPI } from '@/services/api/chatAPI';

interface ChatState {
  conversations: Conversation[];
  currentConversation: Conversation | null;
  messages: Message[];
  isLoading: boolean;
  error: string | null;

  // Actions
  loadConversations: () => Promise<void>;
  loadConversation: (id: string) => Promise<void>;
  createConversation: (title?: string) => Promise<Conversation>;
  deleteConversation: (id: string) => Promise<void>;
  sendMessage: (content: string, type: 'user' | 'assistant') => Promise<void>;
~~~

```
  clearError: () => void;
  setCurrentConversation: (conversation: Conversation | null) => void;
}

export const useChatStore = create<ChatState>()(
  persist(
    (set, get) => ({
      conversations: [],
      currentConversation: null,
      messages: [],
      isLoading: false,
      error: null,

      loadConversations: async () => {
        set({ isLoading: true, error: null });
        try {
          const conversations = await chatAPI.getConversations();
          set({ conversations, isLoading: false });
        } catch (error) {
          set({
            error: error instanceof Error ? error.message : 'Failed to load conversations',
            isLoading: false
          });
        }
      },

      loadConversation: async (id: string) => {
        set({ isLoading: true, error: null });
        try {
          const conversation = await chatAPI.getConversation(id);
          const messages = await chatAPI.getMessages(id);
          set({
            currentConversation: conversation,
            messages,
            isLoading: false
          });
        } catch (error) {
          set({
            error: error instanceof Error ? error.message : 'Failed to load conversation',
            isLoading: false
          });
        }
      },

      createConversation: async (title?: string) => {
        set({ isLoading: true, error: null });
        try {
          const conversation = await chatAPI.createConversation({ title });
          set(state => ({
            conversations: [conversation, ...state.conversations],
            currentConversation: conversation,
            messages: [],
            isLoading: false
          }));
          return conversation;
        } catch (error) {
          set({
            error: error instanceof Error ? error.message : 'Failed to create conversation',
            isLoading: false
          });
          throw error;
        }
      },

      deleteConversation: async (id: string) => {
        set({ isLoading: true, error: null });
        try {
          await chatAPI.deleteConversation(id);
          set(state => ({
            conversations: state.conversations.filter(c => c.id !== id),
            currentConversation: state.currentConversation?.id === id ? null : state.currentConversation,
            messages: state.currentConversation?.id === id ? [] : state.messages,
            isLoading: false
          }));
        } catch (error) {
          set({
            error: error instanceof Error ? error.message : 'Failed to delete conversation',
            isLoading: false
          });
        }
      },

      sendMessage: async (content: string, type: 'user' | 'assistant') => {
        const { currentConversation } = get();
        if (!currentConversation) {
          throw new Error('No active conversation');
        }

        set({ isLoading: true, error: null });
        try {
          const message = await chatAPI.sendMessage(currentConversation.id, {
            content,
            type
          });

          set(state => ({
            messages: [...state.messages, message],
            isLoading: false
          }));
        } catch (error) {
          set({
            error: error instanceof Error ? error.message : 'Failed to send message',
            isLoading: false
          });
          throw error;
        }
      },

      clearError: () => set({ error: null }),
```

~~~
      setCurrentConversation: (conversation) => set({ currentConversation: conversation })
    }),
    {
      name: 'chat-store',
      partialize: (state) => ({
        conversations: state.conversations,
        currentConversation: state.currentConversation
      })
    }
  )
);
```

**Auth Store (src/stores/authStore.ts)**:
```typescript
import { create } from 'zustand';
import { persist } from 'zustand/middleware';
import { User, AuthTokens } from '@/types';
import { authAPI } from '@/services/api/authAPI';

interface AuthState {
  user: User | null;
  tokens: AuthTokens | null;
  isAuthenticated: boolean;
  isLoading: boolean;
  error: string | null;

  // Actions
  login: (email: string, password: string) => Promise<void>;
  register: (userData: RegisterData) => Promise<void>;
  logout: () => void;
  refreshToken: () => Promise<void>;
  updateProfile: (userData: Partial<User>) => Promise<void>;
  clearError: () => void;
}

interface RegisterData {
  email: string;
  password: string;
  name: string;
}

export const useAuthStore = create<AuthState>()(
  persist(
    (set, get) => ({
      user: null,
      tokens: null,
      isAuthenticated: false,
      isLoading: false,
      error: null,

      login: async (email: string, password: string) => {
        set({ isLoading: true, error: null });
        try {
          const response = await authAPI.login({ email, password });
          set({
            user: response.user,
            tokens: response.tokens,
            isAuthenticated: true,
            isLoading: false
          });
        } catch (error) {
          set({
            error: error instanceof Error ? error.message : 'Login failed',
            isLoading: false
          });
          throw error;
        }
      },

      register: async (userData: RegisterData) => {
        set({ isLoading: true, error: null });
        try {
          const response = await authAPI.register(userData);
          set({
            user: response.user,
            tokens: response.tokens,
            isAuthenticated: true,
            isLoading: false
          });
        } catch (error) {
          set({
            error: error instanceof Error ? error.message : 'Registration failed',
            isLoading: false
          });
          throw error;
        }
      },

      logout: () => {
        const { tokens } = get();
        if (tokens) {
          authAPI.logout(tokens.refreshToken).catch(console.error);
        }
        set({
          user: null,
          tokens: null,
          isAuthenticated: false,
          error: null
        });
      },

      refreshToken: async () => {
        const { tokens } = get();
        if (!tokens?.refreshToken) {
          throw new Error('No refresh token available');
        }

        try {
~~~

~~~
          const newTokens = await authAPI.refreshToken(tokens.refreshToken);
          set({ tokens: newTokens });
        } catch (error) {
          // If refresh fails, logout user
          get().logout();
          throw error;
        }
      },

      updateProfile: async (userData: Partial<User>) => {
        set({ isLoading: true, error: null });
        try {
          const updatedUser = await authAPI.updateProfile(userData);
          set({
            user: updatedUser,
            isLoading: false
          });
        } catch (error) {
          set({
            error: error instanceof Error ? error.message : 'Profile update failed',
            isLoading: false
          });
          throw error;
        }
      },

      clearError: () => set({ error: null })
    }),
    {
      name: 'auth-store',
      partialize: (state) => ({
        user: state.user,
        tokens: state.tokens,
        isAuthenticated: state.isAuthenticated
      })
    }
  )
);
```
~~~

Step 4: Performance Optimization and PWA ImplementationI

mplement comprehensive performance optimization and PWA features:

~~~
Create performance optimization and PWA implementation:

**Vite Configuration with Performance Optimization (vite.config.ts)**:
```typescript
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import { resolve } from 'path';
import { visualizer } from 'rollup-plugin-visualizer';
import { VitePWA } from 'vite-plugin-pwa';

export default defineConfig({
  plugins: [
    react(),
    VitePWA({
      registerType: 'autoUpdate',
      workbox: {
        globPatterns: ['**/*.{js,css,html,ico,png,svg,woff2}'],
        runtimeCaching: [
          {
            urlPattern: /^https:\/\/api\.chatbot\.com\/.*$/,
            handler: 'NetworkFirst',
            options: {
              cacheName: 'api-cache',
              expiration: {
                maxEntries: 100,
                maxAgeSeconds: 60 * 60 * 24 // 24 hours
              }
            }
          },
          {
            urlPattern: /^https:\/\/fonts\.googleapis\.com\/.*$/,
            handler: 'CacheFirst',
            options: {
              cacheName: 'google-fonts-cache',
              expiration: {
                maxEntries: 10,
                maxAgeSeconds: 60 * 60 * 24 * 365 // 1 year
              }
            }
          }
        ]
      },
      manifest: {
        name: 'MyCoolChatbot-KiroSpec',
        short_name: 'MyCoolChatbot',
        description: 'AI-powered MyCoolChatbot-KiroSpec application',
        theme_color: '#0ea5e9',
        background_color: '#ffffff',
        display: 'standalone',
        orientation: 'portrait',
        scope: '/',
        start_url: '/',
        icons: [
          {
            src: 'icons/icon-72x72.png',
            sizes: '72x72',
            type: 'image/png'
          },
          {
            src: 'icons/icon-96x96.png',
            sizes: '96x96',
            type: 'image/png'
          },
~~~

~~~
          {
            src: 'icons/icon-128x128.png',
            sizes: '128x128',
            type: 'image/png'
          },
          {
            src: 'icons/icon-144x144.png',
            sizes: '144x144',
            type: 'image/png'
          },
          {
            src: 'icons/icon-152x152.png',
            sizes: '152x152',
            type: 'image/png'
          },
          {
            src: 'icons/icon-192x192.png',
            sizes: '192x192',
            type: 'image/png'
          },
          {
            src: 'icons/icon-384x384.png',
            sizes: '384x384',
            type: 'image/png'
          },
          {
            src: 'icons/icon-512x512.png',
            sizes: '512x512',
            type: 'image/png'
          }
        ]
      }
    }),
    visualizer({
      filename: 'dist/stats.html',
      open: true,
      gzipSize: true
    })
  ],
  resolve: {
    alias: {
      '@': resolve(__dirname, 'src')
    }
  },
  build: {
    target: 'es2020',
    rollupOptions: {
      output: {
        manualChunks: {
          vendor: ['react', 'react-dom'],
          cloudscape: ['@cloudscape-design/components'],
          router: ['react-router-dom'],
          state: ['zustand'],
          utils: ['axios', 'date-fns']
        }
      }
    },
    chunkSizeWarningLimit: 1000
  },
  optimizeDeps: {
    include: [
      'react',
      'react-dom',
      '@cloudscape-design/components',
      '@cloudscape-design/global-styles'
    ]
  }
});
```

**Performance Monitoring Hook (src/hooks/usePerformanceMonitoring.ts)**:
```typescript
import { useEffect } from 'react';

interface PerformanceMetrics {
  lcp: number; // Largest Contentful Paint
  fid: number; // First Input Delay
  cls: number; // Cumulative Layout Shift
  fcp: number; // First Contentful Paint
  ttfb: number; // Time to First Byte
}

export const usePerformanceMonitoring = () => {
  useEffect(() => {
    // Monitor Core Web Vitals
    const observer = new PerformanceObserver((list) => {
      for (const entry of list.getEntries()) {
        switch (entry.entryType) {
          case 'largest-contentful-paint':
            reportMetric('LCP', entry.startTime);
            break;
          case 'first-input':
            reportMetric('FID', (entry as any).processingStart - entry.startTime);
            break;
          case 'layout-shift':
            if (!(entry as any).hadRecentInput) {
              reportMetric('CLS', (entry as any).value);
            }
            break;
        }
      }
    });

    observer.observe({ entryTypes: ['largest-contentful-paint', 'first-input', 'layout-shift'] });

    // Monitor navigation timing
    const navigationObserver = new PerformanceObserver((list) => {
      for (const entry of list.getEntries()) {
        const navEntry = entry as PerformanceNavigationTiming;
~~~

~~~
        reportMetric('FCP', navEntry.responseStart - navEntry.fetchStart);
        reportMetric('TTFB', navEntry.responseStart - navEntry.requestStart);
      }
    });

    navigationObserver.observe({ entryTypes: ['navigation'] });

    return () => {
      observer.disconnect();
      navigationObserver.disconnect();
    };
  }, []);

  const reportMetric = (name: string, value: number) => {
    // Send metrics to analytics service
    if (window.gtag) {
      window.gtag('event', name, {
        event_category: 'Web Vitals',
        value: Math.round(value),
        non_interaction: true
      });
    }

    // Log to console in development
    if (process.env.NODE_ENV === 'development') {
      console.log(`${name}: ${value.toFixed(2)}ms`);
    }
  };
};
```

**Lazy Loading and Code Splitting (src/components/LazyComponents.tsx)**:
```typescript
import { lazy, Suspense } from 'react';
import { Spinner, Box } from '@cloudscape-design/components';

// Lazy load components
export const ChatInterface = lazy(() =>
  import('./chat/ChatInterface').then(module => ({ default: module.ChatInterface }))
);

export const ConversationList = lazy(() =>
  import('./chat/ConversationList').then(module => ({ default: module.ConversationList }))
);

export const UserProfile = lazy(() =>
  import('./user/UserProfile').then(module => ({ default: module.UserProfile }))
);

export const Settings = lazy(() =>
  import('./settings/Settings').then(module => ({ default: module.Settings }))
);

// Loading fallback component
export const LoadingFallback = () => (
  <Box textAlign="center" padding="xxl">
    <Spinner size="large" />
  </Box>
);

// HOC for lazy loading with Suspense
export const withLazyLoading = <P extends object>(
  Component: React.ComponentType<P>
) => {
  return (props: P) => (
    <Suspense fallback={<LoadingFallback />}>
      <Component {...props} />
    </Suspense>
  );
};
```

**Image Optimization Hook (src/hooks/useImageOptimization.ts)**:
```typescript
import { useState, useEffect } from 'react';

interface ImageOptimizationOptions {
  width?: number;
  height?: number;
  quality?: number;
  format?: 'webp' | 'avif' | 'jpeg' | 'png';
  lazy?: boolean;
}

export const useImageOptimization = (
  src: string,
  options: ImageOptimizationOptions = {}
) => {
  const [optimizedSrc, setOptimizedSrc] = useState<string>('');
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    const optimizeImage = async () => {
      try {
        setIsLoading(true);
        setError(null);

        // Check if browser supports modern formats
        const supportsWebP = await checkWebPSupport();
        const supportsAVIF = await checkAVIFSupport();

        let format = options.format;
        if (!format) {
          if (supportsAVIF) format = 'avif';
          else if (supportsWebP) format = 'webp';
          else format = 'jpeg';
        }
~~~

~~~
        // Build optimized URL (assuming image optimization service)
        const params = new URLSearchParams();
        if (options.width) params.set('w', options.width.toString());
        if (options.height) params.set('h', options.height.toString());
        if (options.quality) params.set('q', options.quality.toString());
        params.set('f', format);

        const optimizedUrl = `${src}?${params.toString()}`;
        setOptimizedSrc(optimizedUrl);
      } catch (err) {
        setError(err instanceof Error ? err.message : 'Image optimization failed');
        setOptimizedSrc(src); // Fallback to original
      } finally {
        setIsLoading(false);
      }
    };

    if (src) {
      optimizeImage();
    }
  }, [src, options.width, options.height, options.quality, options.format]);

  return { optimizedSrc, isLoading, error };
};

// Helper functions
const checkWebPSupport = (): Promise<boolean> => {
  return new Promise((resolve) => {
    const webP = new Image();
    webP.onload = webP.onerror = () => resolve(webP.height === 2);
    webP.src = 'data:image/webp;base64,UklGRjoAAABXRUJQVlA4IC4AAACyAgCdASoCAAIALmk0mk0iIiIiIgBoSygABc6WWgAA/veff/0PP8bA//LwYAAA';
  });
};

const checkAVIFSupport = (): Promise<boolean> => {
  return new Promise((resolve) => {
    const avif = new Image();
    avif.onload = avif.onerror = () => resolve(avif.height === 2);
    avif.src = 'data:image/avif;base64,AAAAIGZ0eXBhdmlmAAAAAGF2aWZtaWYxbWlhZk1BMUIAAADybWV0YQAAAAAAAAAoaGRscgAAAAAAAAAAcGljdAAAAAAAAAAAAAAAAGxpYmF2aWYAAAAADnBpdG0AAAAAAAEAAAAea
  });
};
```
~~~

Step 5: Accessibility and Testing ImplementationImplemen

t comprehensive accessibility and testing:

~~~
Create accessibility and testing implementation:

**Accessibility Hook (src/hooks/useAccessibility.ts)**:
```typescript
import { useEffect, useState } from 'react';

interface AccessibilityPreferences {
  reducedMotion: boolean;
  highContrast: boolean;
  fontSize: 'small' | 'medium' | 'large';
  screenReader: boolean;
}

export const useAccessibility = () => {
  const [preferences, setPreferences] = useState<AccessibilityPreferences>({
    reducedMotion: false,
    highContrast: false,
    fontSize: 'medium',
    screenReader: false
  });

  useEffect(() => {
    // Detect user preferences
    const mediaQueries = {
      reducedMotion: window.matchMedia('(prefers-reduced-motion: reduce)'),
      highContrast: window.matchMedia('(prefers-contrast: high)'),
      screenReader: window.navigator.userAgent.includes('NVDA') ||
                   window.navigator.userAgent.includes('JAWS') ||
                   window.speechSynthesis !== undefined
    };

    const updatePreferences = () => {
      setPreferences({
        reducedMotion: mediaQueries.reducedMotion.matches,
        highContrast: mediaQueries.highContrast.matches,
        fontSize: localStorage.getItem('fontSize') as any || 'medium',
        screenReader: mediaQueries.screenReader
      });
    };

    // Initial check
    updatePreferences();

    // Listen for changes
    mediaQueries.reducedMotion.addEventListener('change', updatePreferences);
    mediaQueries.highContrast.addEventListener('change', updatePreferences);

    return () => {
      mediaQueries.reducedMotion.removeEventListener('change', updatePreferences);
      mediaQueries.highContrast.removeEventListener('change', updatePreferences);
    };
  }, []);

  const updateFontSize = (size: 'small' | 'medium' | 'large') => {
    setPreferences(prev => ({ ...prev, fontSize: size }));
    localStorage.setItem('fontSize', size);

    // Apply font size to document
~~~

~~~
    const fontSizeMap = {
      small: '14px',
      medium: '16px',
      large: '18px'
    };
    document.documentElement.style.fontSize = fontSizeMap[size];
  };

  const announceToScreenReader = (message: string) => {
    const announcement = document.createElement('div');
    announcement.setAttribute('aria-live', 'polite');
    announcement.setAttribute('aria-atomic', 'true');
    announcement.style.position = 'absolute';
    announcement.style.left = '-10000px';
    announcement.style.width = '1px';
    announcement.style.height = '1px';
    announcement.style.overflow = 'hidden';

    document.body.appendChild(announcement);
    announcement.textContent = message;

    setTimeout(() => {
      document.body.removeChild(announcement);
    }, 1000);
  };

  return {
    preferences,
    updateFontSize,
    announceToScreenReader
  };
};
```

**Keyboard Navigation Hook (src/hooks/useKeyboardNavigation.ts)**:
```typescript
import { useEffect, useRef } from 'react';

interface KeyboardNavigationOptions {
  trapFocus?: boolean;
  autoFocus?: boolean;
  onEscape?: () => void;
  onEnter?: () => void;
}

export const useKeyboardNavigation = (options: KeyboardNavigationOptions = {}) => {
  const containerRef = useRef<HTMLDivElement>(null);
  const { trapFocus, autoFocus, onEscape, onEnter } = options;

  useEffect(() => {
    const container = containerRef.current;
    if (!container) return;

    // Auto focus first focusable element
    if (autoFocus) {
      const firstFocusable = container.querySelector(
        'button, [href], input, select, textarea, [tabindex]:not([tabindex="-1"])'
      ) as HTMLElement;
      firstFocusable?.focus();
    }

    const handleKeyDown = (event: KeyboardEvent) => {
      // Handle Escape key
      if (event.key === 'Escape' && onEscape) {
        onEscape();
        return;
      }

      // Handle Enter key
      if (event.key === 'Enter' && onEnter) {
        onEnter();
        return;
      }

      // Handle Tab key for focus trapping
      if (event.key === 'Tab' && trapFocus) {
        const focusableElements = container.querySelectorAll(
          'button, [href], input, select, textarea, [tabindex]:not([tabindex="-1"])'
        ) as NodeListOf<HTMLElement>;

        const firstElement = focusableElements[0];
        const lastElement = focusableElements[focusableElements.length - 1];

        if (event.shiftKey) {
          // Shift + Tab
          if (document.activeElement === firstElement) {
            event.preventDefault();
            lastElement.focus();
          }
        } else {
          // Tab
          if (document.activeElement === lastElement) {
            event.preventDefault();
            firstElement.focus();
          }
        }
      }
    };

    container.addEventListener('keydown', handleKeyDown);
    return () => container.removeEventListener('keydown', handleKeyDown);
  }, [trapFocus, autoFocus, onEscape, onEnter]);

  return containerRef;
};
```

**Comprehensive Test Suite (src/tests/components/ChatInterface.test.tsx)**:
```typescript
~~~

```
import React from 'react';
import { render, screen, fireEvent, waitFor } from '@testing-library/react';
import { axe, toHaveNoViolations } from 'jest-axe';
import userEvent from '@testing-library/user-event';
import { ChatInterface } from '@/components/chat/ChatInterface';
import { useChatStore } from '@/stores/chatStore';

// Extend Jest matchers
expect.extend(toHaveNoViolations);

// Mock stores
jest.mock('@/stores/chatStore');
const mockUseChatStore = useChatStore as jest.MockedFunction<typeof useChatStore>;

// Mock AI service
jest.mock('@/hooks/useAIService', () => ({
  useAIService: () => ({
    generateResponse: jest.fn().mockResolvedValue('Mocked AI response')
  })
}));

describe('ChatInterface', () => {
  const mockStore = {
    currentConversation: {
      id: '1',
      title: 'Test Conversation',
      userId: 'user1',
      messages: [],
      createdAt: new Date(),
      updatedAt: new Date(),
      metadata: {
        totalMessages: 0,
        totalTokens: 0,
        lastActivity: new Date(),
        tags: []
      }
    },
    messages: [],
    sendMessage: jest.fn(),
    isLoading: false,
    error: null
  };

  beforeEach(() => {
    mockUseChatStore.mockReturnValue(mockStore as any);
  });

  afterEach(() => {
    jest.clearAllMocks();
  });

  describe('Rendering', () => {
    it('renders chat interface correctly', () => {
      render(<ChatInterface />);

      expect(screen.getByText('Chat Interface')).toBeInTheDocument();
      expect(screen.getByPlaceholderText('Type your message...')).toBeInTheDocument();
      expect(screen.getByRole('button', { name: /send message/i })).toBeInTheDocument();
    });

    it('displays conversation title', () => {
      render(<ChatInterface />);

      expect(screen.getByText('Test Conversation')).toBeInTheDocument();
    });

    it('shows message count badge', () => {
      render(<ChatInterface />);

      expect(screen.getByText('0 messages')).toBeInTheDocument();
    });
  });

  describe('Accessibility', () => {
    it('should not have accessibility violations', async () => {
      const { container } = render(<ChatInterface />);
      const results = await axe(container);
      expect(results).toHaveNoViolations();
    });

    it('has proper ARIA labels', () => {
      render(<ChatInterface />);

      expect(screen.getByLabelText('Chat message input')).toBeInTheDocument();
      expect(screen.getByLabelText('Send message')).toBeInTheDocument();
    });

    it('supports keyboard navigation', async () => {
      const user = userEvent.setup();
      render(<ChatInterface />);

      const input = screen.getByPlaceholderText('Type your message...');
      const sendButton = screen.getByRole('button', { name: /send message/i });

      // Tab to input
      await user.tab();
      expect(input).toHaveFocus();

      // Tab to send button
      await user.tab();
      expect(sendButton).toHaveFocus();
    });

    it('handles Enter key to send message', async () => {
      const user = userEvent.setup();
      render(<ChatInterface />);

      const input = screen.getByPlaceholderText('Type your message...');
```

```
      await user.type(input, 'Test message');
      await user.keyboard('{Enter}');

      expect(mockStore.sendMessage).toHaveBeenCalledWith('Test message', 'user');
    });
  });

  describe('User Interactions', () => {
    it('sends message when send button is clicked', async () => {
      const user = userEvent.setup();
      render(<ChatInterface />);

      const input = screen.getByPlaceholderText('Type your message...');
      const sendButton = screen.getByRole('button', { name: /send message/i });

      await user.type(input, 'Hello, AI!');
      await user.click(sendButton);

      expect(mockStore.sendMessage).toHaveBeenCalledWith('Hello, AI!', 'user');
    });

    it('disables send button when input is empty', () => {
      render(<ChatInterface />);

      const sendButton = screen.getByRole('button', { name: /send message/i });
      expect(sendButton).toBeDisabled();
    });

    it('enables send button when input has text', async () => {
      const user = userEvent.setup();
      render(<ChatInterface />);

      const input = screen.getByPlaceholderText('Type your message...');
      const sendButton = screen.getByRole('button', { name: /send message/i });

      await user.type(input, 'Test');
      expect(sendButton).toBeEnabled();
    });

    it('clears input after sending message', async () => {
      const user = userEvent.setup();
      render(<ChatInterface />);

      const input = screen.getByPlaceholderText('Type your message...');
      const sendButton = screen.getByRole('button', { name: /send message/i });

      await user.type(input, 'Test message');
      await user.click(sendButton);

      expect(input).toHaveValue('');
    });
  });

  describe('Loading States', () => {
    it('shows loading state when sending message', () => {
      mockUseChatStore.mockReturnValue({
        ...mockStore,
        isLoading: true
      } as any);

      render(<ChatInterface />);

      expect(screen.getByText('Processing...')).toBeInTheDocument();
      expect(screen.getByPlaceholderText('Type your message...')).toBeDisabled();
    });

    it('shows typing indicator when AI is responding', () => {
      render(<ChatInterface />);

      // Simulate typing state (this would be managed by the component)
      // For this test, we'll check if the component can handle the typing state
      expect(screen.queryByText('AI is typing...')).not.toBeInTheDocument();
    });
  });

  describe('Error Handling', () => {
    it('displays error message when present', () => {
      mockUseChatStore.mockReturnValue({
        ...mockStore,
        error: 'Failed to send message'
      } as any);

      render(<ChatInterface />);

      expect(screen.getByText('Failed to send message')).toBeInTheDocument();
    });

    it('allows dismissing error alerts', async () => {
      const user = userEvent.setup();
      mockUseChatStore.mockReturnValue({
        ...mockStore,
        error: 'Test error'
      } as any);

      render(<ChatInterface />);

      const dismissButton = screen.getByRole('button', { name: /dismiss/i });
      await user.click(dismissButton);

      // Error should be cleared (this would trigger clearError in real implementation)
    });
  });

  describe('Message Display', () => {
    it('renders messages correctly', () => {
      const messagesWithData = [
        {
          id: '1',
```

```
          conversationId: '1',
          content: 'Hello!',
          type: 'user' as const,
          timestamp: new Date(),
          metadata: { tokens: 5 }
        },
        {
          id: '2',
          conversationId: '1',
          content: 'Hi there!',
          type: 'assistant' as const,
          timestamp: new Date(),
          metadata: { tokens: 10 }
        }
      ];

      mockUseChatStore.mockReturnValue({
        ...mockStore,
        messages: messagesWithData
      } as any);

      render(<ChatInterface />);

      expect(screen.getByText('Hello!')).toBeInTheDocument();
      expect(screen.getByText('Hi there!')).toBeInTheDocument();
    });

    it('scrolls to bottom when new messages arrive', () => {
      const scrollIntoViewMock = jest.fn();
      Element.prototype.scrollIntoView = scrollIntoViewMock;

      render(<ChatInterface />);

      // This would be tested with a more complex setup that simulates message updates
      expect(scrollIntoViewMock).toHaveBeenCalled();
    });
  });
});
```

E2E Test with Playwright (tests/e2e/chat-flow.spec.ts):

```typescript
import { test, expect } from '@playwright/test';

test.describe('Chat Flow', () => {
  test.beforeEach(async ({ page }) => {
    // Mock API responses
    await page.route('**/api/auth/profile', async route => {
      await route.fulfill({
        json: {
          user: {
            id: '1',
            name: 'Test User',
            email: 'test@example.com'
          }
        }
      });
    });

    await page.route('**/api/conversations', async route => {
      await route.fulfill({
        json: {
          conversations: []
        }
      });
    });

    await page.goto('/chat');
  });

  test('should load chat interface', async ({ page }) => {
    await expect(page.getByText('Chat Interface')).toBeVisible();
    await expect(page.getByPlaceholder('Type your message...')).toBeVisible();
    await expect(page.getByRole('button', { name: /send message/i })).toBeVisible();
  });

  test('should send and receive messages', async ({ page }) => {
    // Mock AI response
    await page.route('**/api/ai/chat', async route => {
      await route.fulfill({
        json: {
          message: {
            id: '2',
            content: 'Hello! How can I help you today?',
            type: 'assistant',
            timestamp: new Date().toISOString()
          }
        }
      });
    });

    const messageInput = page.getByPlaceholder('Type your message...');
    const sendButton = page.getByRole('button', { name: /send message/i });

    // Type and send message
    await messageInput.fill('Hello, AI!');
    await sendButton.click();

    // Check that message appears
    await expect(page.getByText('Hello, AI!')).toBeVisible();
    await expect(page.getByText('Hello! How can I help you today?')).toBeVisible();

    // Check that input is cleared
    await expect(messageInput).toHaveValue('');
  });

  test('should handle keyboard navigation', async ({ page }) => {
    const messageInput = page.getByPlaceholder('Type your message...');
    const sendButton = page.getByRole('button', { name: /send message/i });
```

~~~
    // Tab navigation
    await page.keyboard.press('Tab');
    await expect(messageInput).toBeFocused();

    await page.keyboard.press('Tab');
    await expect(sendButton).toBeFocused();
  });

  test('should send message with Enter key', async ({ page }) => {
    const messageInput = page.getByPlaceholder('Type your message...');

    await messageInput.fill('Test message');
    await messageInput.press('Enter');

    // Verify message was sent (would check for API call or message appearance)
    await expect(page.getByText('Test message')).toBeVisible();
  });

  test('should be accessible', async ({ page }) => {
    // Check for proper ARIA labels
    await expect(page.getByLabel('Chat message input')).toBeVisible();
    await expect(page.getByLabel('Send message')).toBeVisible();

    // Check color contrast (basic check)
    const messageInput = page.getByPlaceholder('Type your message...');
    const styles = await messageInput.evaluate(el => {
      const computed = window.getComputedStyle(el);
      return {
        color: computed.color,
        backgroundColor: computed.backgroundColor
      };
    });

    // Basic contrast check (would use more sophisticated contrast checking in real tests)
    expect(styles.color).toBeTruthy();
    expect(styles.backgroundColor).toBeTruthy();
  });

  test('should work with reduced motion preference', async ({ page }) => {
    // Set reduced motion preference
    await page.emulateMedia({ reducedMotion: 'reduce' });

    await page.reload();

    // Verify that animations are disabled or reduced
    // This would check for specific CSS classes or animation properties
    const chatContainer = page.locator('[data-testid="chat-container"]');
    await expect(chatContainer).toHaveCSS('animation-duration', '0s');
  });
});
```
~~~

Frontend Implementation Complete

Your Frontend phase implementation is now complete with:

✅ Implemented Components

- **AWS Cloudscape Design System** with comprehensive UI components and theming
- **React Design Systems MCP Server** integration for best practices and guidance
- **Complete chat interface** with real-time messaging and AI integration
- **State management** with Zustand and persistent storage
- **Performance optimization** with lazy loading, code splitting, and PWA features

✅ Quality Assurance

- **Comprehensive accessibility** with WCAG 2.1 AA compliance and screen reader support
- **Extensive testing** with unit tests, integration tests, and E2E tests
- **Performance monitoring** with Core Web Vitals tracking and optimization
- **Cross-browser compatibility** with modern web standards

✅ Kiro Features Integration

- **React Design Systems MCP Server**: AWS Cloudscape best practices and component guidance
- **Performance optimization**: Bundle analysis and optimization recommendations
- **Accessibility validation**: Automated accessibility testing and compliance checking
- **Code quality**: Automated testing and documentation updates

🚀 Next Steps

With your frontend complete, you're ready to:

1. **Integrate with backend APIs** - Connect to the Lambda functions and DynamoDB
2. **Deploy to CloudFront** - Serve the optimized frontend through AWS CDN
3. **Monitor performance** - Track Core Web Vitals and user experience metrics
4. **Iterate based on feedback** - Use analytics to improve user experience

Your frontend provides a modern, accessible, and performant user interface built with AWS Cloudscape Design System and enhanced by Kiro's react-design-systems-mcp server guidance.

Previous Next